StampMyVisa

HIRING ASSIGNMENT

AI Product Manager

> **Time limit: 4 hours**
> Report your actual start and end time.

The Setup

TiffinLoop is a meal subscription marketplace connecting home cooks with working professionals. Subscribers get daily lunch or dinner from a cook assigned to them, delivered by local gig delivery partners.

The scale: 500+ subscribers, 90+ home cooks, across Bengaluru, Mumbai, and Pune.

Everything today runs on WhatsApp, spreadsheets, and phone calls. There is no engineering team available to you.

The Problem You're Solving

Home cooks drop out with little to no notice. A cook falls sick, has a family emergency, or simply doesn't show up.

When this happens today: the ops team finds out late, usually when a subscriber complains. They scramble on WhatsApp to find a backup cook or issue a refund. Subscribers frequently get zero communication until their meal simply doesn't arrive.

Build a tool that solves this: when a cook drops out, the system should help ops quickly figure out who is affected, what the fallback options are, and make sure every affected subscriber gets notified before their meal time, not after.

The Data

Attached: tiffinloop_seed.zip. Four files: cooks, subscribers, and orders exported from TiffinLoop's hand-maintained sheets, plus this morning's export of the ops team WhatsApp group.

It is 10:30 AM. Lunch goes out at 12:30 PM and dinner at 7:30 PM. Nothing dated today has been delivered yet.

You must build against this data as given. Handling its inconsistencies is part of the job, and doing that in code is exactly what we want to see. What's not allowed: fixing the files by hand, or swapping in cleaner data you made up.

> Your build will be tested against real scenarios drawn from this data, including some that are not obvious on a first read. Look closely before you start building.

Success Criteria

1. **Speed of resolution:** when a cook drops out, how fast can the tool surface every affected subscriber and their order details?
2. **Fallback logic:** does the tool help identify realistic backup options, and does it correctly handle conflicts when a backup cannot cover everyone affected?
3. **Subscriber communication:** does an affected subscriber actually get notified, with a message containing what they need to know? Notifications can be simulated: show the message and whether it was sent. You don't need a real SMS or WhatsApp integration.
4. **Traceability:** can ops look back and see what happened for a given dropout event: who was affected, what was decided, was everyone notified?

You do not need to solve all four perfectly in 4 hours. You need to show us how you'd prioritize within this constraint, which parts you built deeply vs. shallowly, and why.

The Second Build (required)

In addition to the ops-facing tool above, build a one-page leadership view: a simple dashboard pulling from the same data, showing cook dropout frequency by city and by cook over the last 30 days.

This is a different user with a different need: leadership isn't resolving today's dropout, they're spotting a pattern. Build for that.

What You'll Submit

1. The Deployed Prototype

A live, working link (Vercel, Railway, Replit, or similar). Must include both the ops tool and the leadership view. We will test it ourselves without any setup help from you.

2. The Code Repository

A link to a public repository (GitHub, GitLab, or similar) containing all the code behind your deployed prototype. Keep it public until we confirm your review is complete. We will read the code to understand how your tool makes its decisions.

Do not commit API keys, passwords, or any other secrets. A repository with exposed secrets counts against you.

3. The PRD (one page, hard limit)

Write this after you build, not before. A one-page PRD for the production version of your tool: the problem, who it's for, how you'd measure success, what your prototype proved, what you deliberately cut and why, and the open questions you'd need answered before an engineer builds it for real.

Anything beyond one page will not be read.

4. The Build Log

Your full prompt history AND the AI's responses. Claude Code, Cursor, ChatGPT, Replit, anything. We want to see how you directed the tool, what you accepted, and what you pushed back on or corrected. Incomplete logs with only your prompts, and not the AI's responses, will not be accepted.

Share it as a link: an exported chat, a Google Doc, or a file in your repository.

Rules

1. **Build against the provided seed data,** not your own fabricated clean version.
2. **The prototype must be usable by a stranger** with zero explanation from you.
3. **The repository must be public** and match the deployed prototype.
4. **The PRD is one page.** Not one page plus an appendix.
5. **Work alone.** The prompts should be yours. The thinking should be yours.
6. Report your actual start and end time.

Submission

Email your four submissions (Prototype link, Repository link, PRD, Build Log) to **mehvish@stampmyvisa.com** with the subject line:

> **AI Product Manager | [Your Name]**

Deadline: check email

Shortlisted candidates will be invited to a second, live assignment as part of the process.